Cairo University - Faculty Of Engineering
Computer Engineering Department
CMP4040 – Machine Intelligence - Spring 2026

TEAM 5 Final Report
League of Legends Match Outcome Prediction

Name	Sec	BN	Email
Ahmed Hamed Gaber Hamed	1	3	ahmed.hamed03@eng-st.cu.edu.eg
Akram Hany Karam Salam	1	13	akram.sallam03@eng-st.cu.edu.eg
Amir Anwar Bekhit Awd Kedis	1	14	amir.awd03@eng-st.cu.edu.eg
Somia Saad Ismail El-Shemy	1	26	somia.elshemy02@eng-st.cu.edu.eg

Date: 2026-04-24

Contribution Summary

- Ahmed Hamed : Pre-processing / MLOps
- Akram Hany : Feature Engineering
- Amir Anwar : Modeling
- Somia Saad : EDA

1 Problem Definition

Predicts the winner of a League of Legends ranked match using machine learning. It supports two related tasks (scopes):

- Post-game analysis: explain the winner from full team-level match statistics.
- Pre-game prediction: estimate the winner from draft-side context and historical champion priors only. *(This one is a secodary scope we added for comparison sake, it wasn't required but it shows a point that we will go through in some detail)*

The problem is non-trivial because the dataset is large, highly structured, and strongly collinear. Economy, combat, and objective features such as gold, kills, assists, turret pressure, and inhibitor control overlap heavily, so model performance must be interpreted carefully.

Figure 1: League of Legends Match Outcome prediction

2 Motivation

This project uses a real dataset, applies multiple non-deep-learning methods, and requires explicit handling of generalization, leakage, and overfitting. It is also meaningful from a game-analysis perspective: the pre-game branch asks how much the draft alone can explain, while the post-game branch identifies which match signals are most associated with winning.

3 Dataset And References

- Dataset: League of Legends Ranked Matches by Paolo Campanelli
- Link: https://www.kaggle.com/datasets/paololol/league-of-legends-ranked-matches/data
- Size: about 184,070 matches (maps to 1.8M participants rows or 368K teams) *each team has 10 particpants each match has 2 teams*
  - we worked on team level so around 368K row
- Target: binary `win` label

4 Evaluation Metrics

- Accuracy (used for tune/eval)
- Precision
- Recall
- F1 score
- ROC-AUC (used for tune/eval)
- PR-AUC
- Brier score
- Log loss

5 Workflow / Actions we did

1. Data Exploration
   - Schema overview
   - Target Distribution (Balanced)
   - Uni-variate and Bi-variate Analysis
   - correlation analysis (Pearson, VIF)

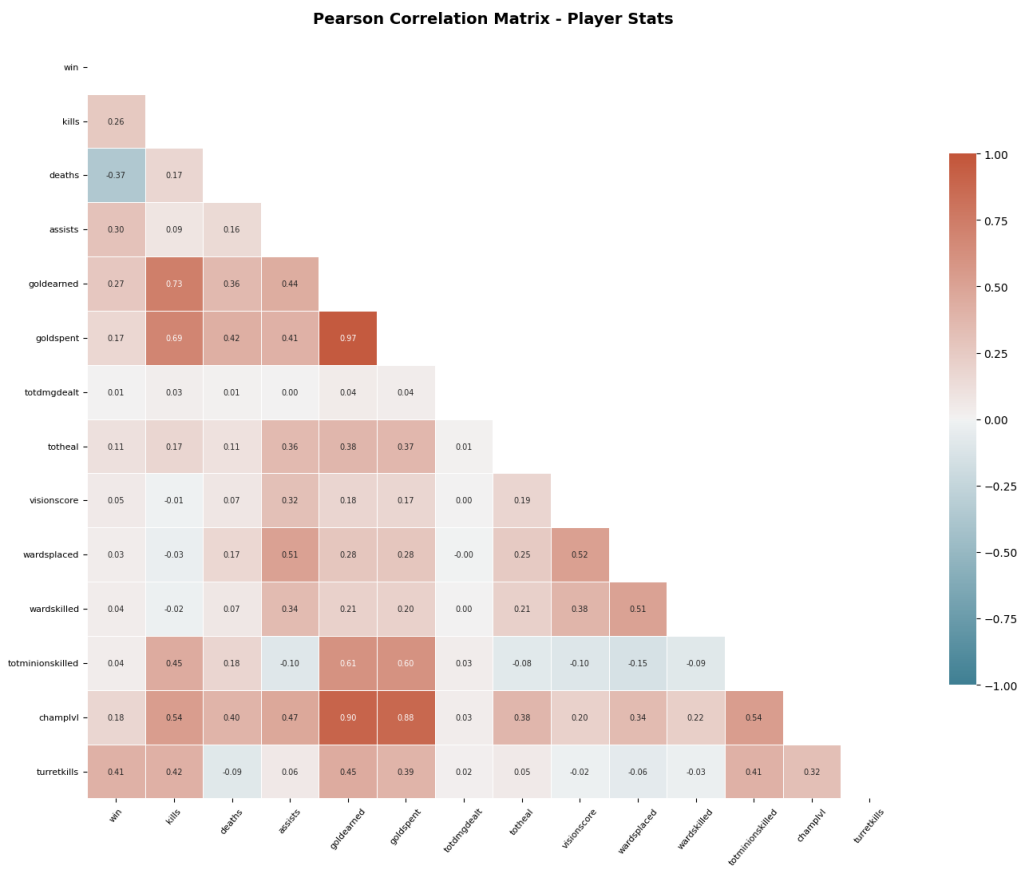

Figure 2: Pearson Correlation Matrix for player stats

- Missing values (There is no missing value on the dataset)

- Champion win rate

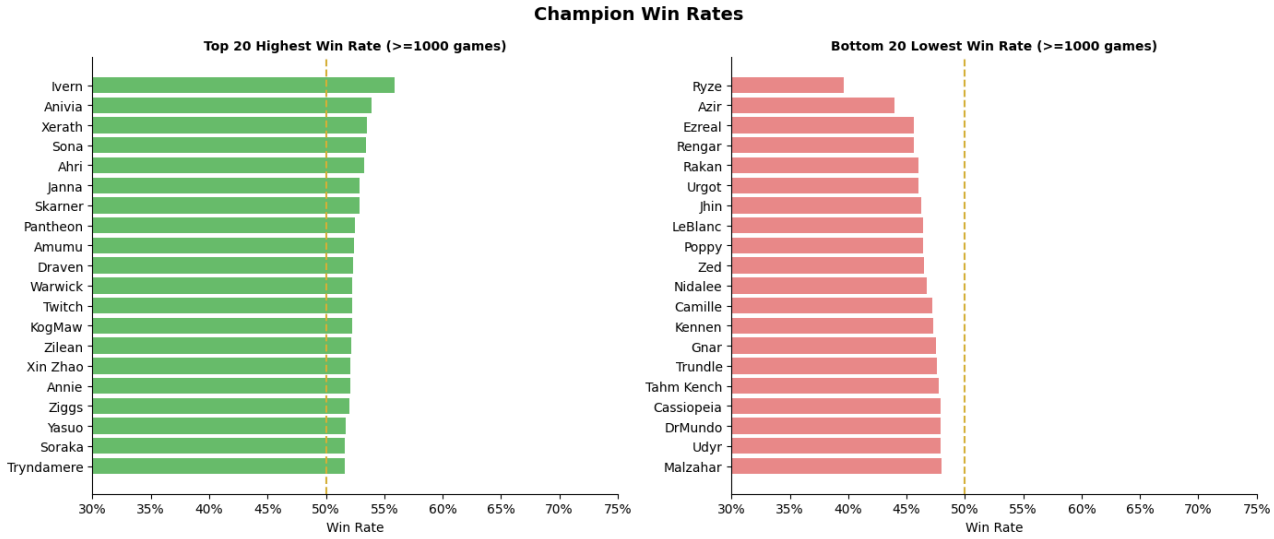

Figure 3: Champions win rate

2. Data Quality and Cleaning
   - Create **Great Exceptions** suites
   - Removed low quality matches (very short in time)
3. Feature engineering
   - Built post-game and pre-game feature sets (added a lot of aggregated features and metrics based on LoL nature and overall game performance metrics).
   - Reduced redundant features using **correlation** and **VIF** filtering
   - Created pre-game champion features for draft-side prediction
4. Training, Tuning, and Validation
   - Split data by `matchid` to prevent complementary-row leakage (each match has 2 rows)
   - Fit preprocessing (encoders) on train only and applied the same transforms to validation/test
   - Trained multiple baseline models first then Advanced Models
   - Performed hyperparameter optimization for best models (using **Optuna** tuning)
   - used **MLFlow** to track all experiments and Optuna trials
5. Analysis of results, leakage, Ablation.
   - Tree based feature importance and SHAP based feature importance
   - Full progressive Ablation analysis to detect if features are leakage of the target or just helpful and how the model functions when removed

6 Results

Dataset Analysis Results

- Raw matches: 184,069
- Team-level rows after cleaning: 358,534, corresponding to 179,267 unique matches
- Target balance: exactly 50% wins and 50% losses in the cleaned team-level dataset
- Missing values after cleaning: 0

The dataset analysis also confirmed the main modeling challenge: strong redundancy among economy, combat, and objective statistics. In the post-game pipeline, 142 numeric candidate features were reduced to 35 final selected features after correlation and VIF filtering. In the pre-game pipeline, 33 candidate features were reduced to 7 final selected features.

Models Used

- Perceptron
- Logistic Regression (L2 regularization)
- Linear Regression (ridge)
- Linear SVM (L2 regularization)
- RBF SVM (L2 regularization)
- Random Forest
- XGBoost (L2 regularization)
- LightGBM

Experimental Results

The final post-game test results are:

Model	Accuracy	ROC-AUC
LightGBM	98.18%	99.86%
XGBoost	98.17%	99.86%
Logistic Regression	97.80%	99.75%
Perceptron	96.93%	99.07%
Linear SVM	86.34%	96.76%
Random Forest	96.95%	99.85%
RBF SVM	97.67%	99.77%

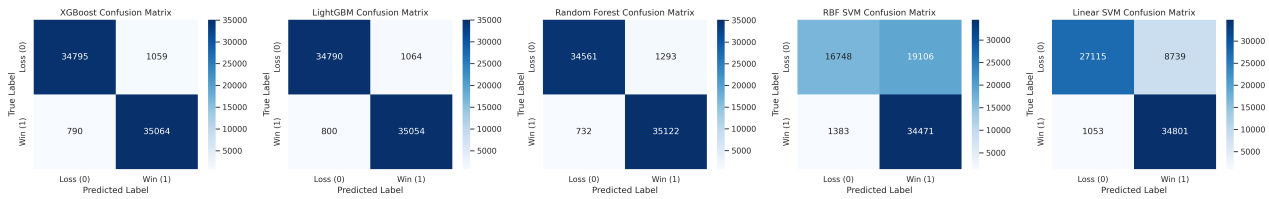

Figure 4: Confusion Matrix for some of the models

The 2 scope comparisons shows the gap between post-game modeling and pre-game prediction based on the draft:

Scope	Model	Accuracy	ROC-AUC
Postgame	LightGBM	98.18%	99.86%
Pregame	LightGBM	55.07%	56.96%

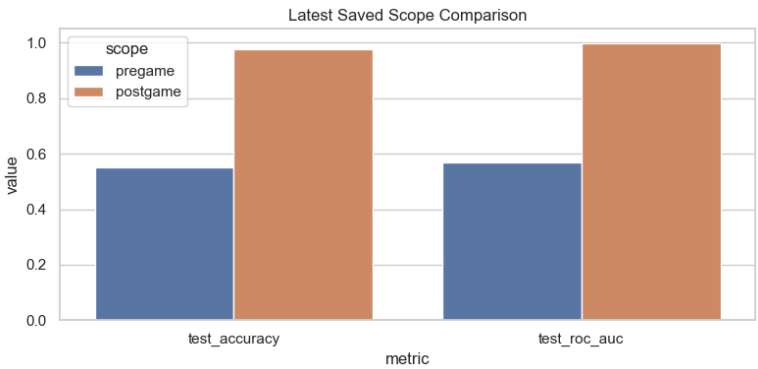

Figure 5: Pregame vs Postgame

- The low accuarcy of the pre-game modling might indicate that RIOT balanced the champoins and draft enough that it doesn't affect the win rate that much.

The champion-model ablation result is:

Setting	Accuracy	ROC-AUC
LightGBM full post-game feature set	98.18%	99.86%
LightGBM with selected objective features removed	97.66%	99.78%

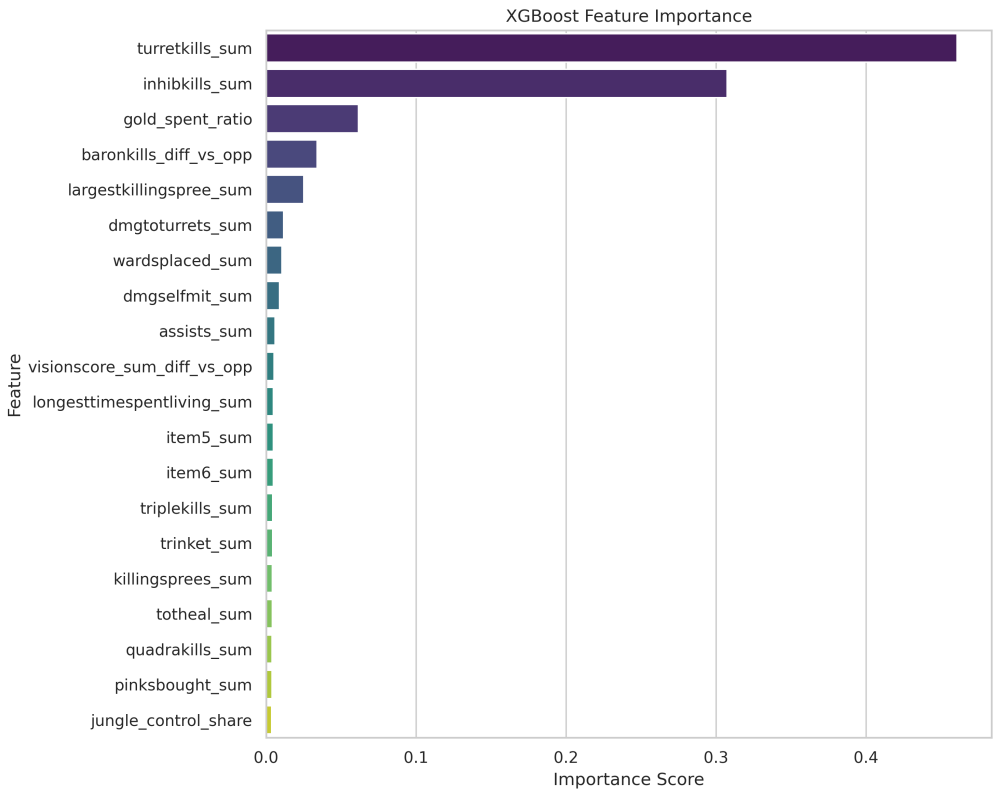

Figure 6: Feature Importance

- After we removed features like turretkills and inhibkills the accuracy didn't drop much indicating that the model doesn't just depend on these features.

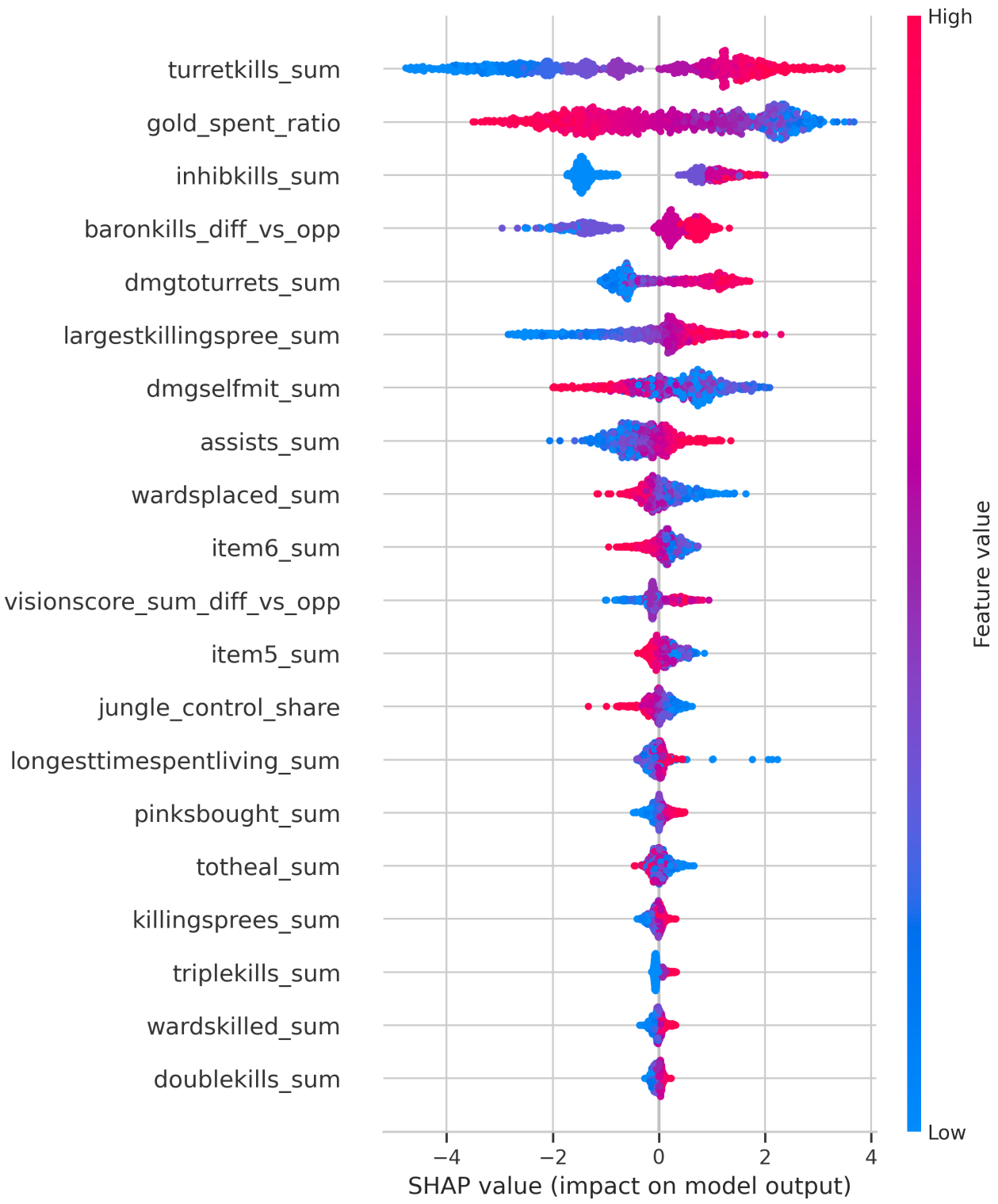

Figure 7: SHAP

These shows that the most important features to win are:

- turret and inhib kills
- Gold features (economy)
- Damage and assits features
- features realted to vision and map coverage like wards placed and visionscore